

EvoLab-as-a-Service

André Vaz, nº 51585, email: A51585@alunos.isel.pt, tel.: 935248411
Miguel Pinto, nº 51557, email: A51557@alunos.isel.pt, tel.: 918108102

Supervisor:

José Simão, email: jose.simao@isel.pt

March 2026

1 Context

Recent advances in Artificial Intelligence and automated optimization have enabled systems not only to execute algorithms, but also to iteratively improve them through experimentation. These approaches automatically generate variations of an algorithm, execute them, and evaluate their performance according to predefined criteria.

This paradigm is formalized in Barbarians at the Gate: How AI is Upending Systems Research [1], which introduces AI-Driven Research for Systems (ADRS), where AI assists in generating and refining candidate solutions through automated evaluation. Frameworks such as OpenEvolve [2] implement this idea in practice.

However, despite their potential, these tools remain largely restricted to technically advanced users. Environment setup, execution management, and result tracking often require manual configuration and specialized knowledge.

There is therefore a gap between the availability of technologies for automatic algorithm evolution and the absence of a structured infrastructure that centralizes, simplifies, and organizes this process.

2 Proposed Solution

To address the identified problem, this project proposes the design and development of a web-based platform dedicated to the structured management of algorithm evolution experiments.

The system will provide an integrated environment where users can:

- Submit an initial version of an algorithm.
- Define an evaluation function that determines how the algorithm’s performance is measured.
- Launch controlled experimental cycles in which new algorithm variations are automatically generated and evaluated.
- Monitor and compare results across different iterations.

Instead of requiring users to manually configure frameworks, manage execution environments, and track results independently, the platform will centralize and orchestrate the entire process. It will serve as an intermediary layer between the user and the underlying evolutionary and AI-based mechanisms.

The proposed solution does not aim to develop a new evolutionary algorithm from scratch. Rather, it focuses on building a structured infrastructure around existing algorithm evolution techniques, integrating them into a coherent system with clear workflows, experiment tracking, and reproducibility guarantees.

By abstracting technical complexity and providing a controlled execution environment, the platform aims to transform algorithm evolution from a fragmented technical task into a structured and manageable process.

3 Requirements

3.1 Functional Requirements

The system must satisfy the following functional requirements:

- Allow creation and management of experimental projects.
- Allow submission of an initial algorithm and its associated evaluation function.
- Support configuration of evolutionary parameters.
- Store generated algorithm versions, execution outputs, and performance metrics.
- Provide visualization and comparison of experimental results.

These requirements define the observable behavior of the platform from a user perspective.

3.2 Non-Functional Requirements

In addition to functional capabilities, the system must satisfy key quality attributes:

- Execution Isolation – Algorithms execute in controlled environments.
- Data Persistence – Experimental data is reliably stored and retrievable.
- Resource Control – Execution time and computational limits are configurable.
- Basic Security – Authenticated user access.
- Modularity – Clear separation between orchestration layer, execution engine, and interface.

These quality constraints ensure reliability, control, and architectural robustness.

4 Architecture

- Frontend: The frontend will be developed using React with TypeScript, ensuring a modern component-based architecture and strong type safety.
- Backend: The backend will be built using Kotlin with Spring Boot, enabling robust REST API development, dependency injection, and clear service-layer architecture.
- Database: The system will use PostgreSQL, a reliable relational database that ensures transactional consistency and supports structured data and complex queries.
- Evolution Execution Layer: The evolutionary optimization process will be executed through a pool of backend workers. Each worker launches an isolated Docker container running the OpenEvolve framework to perform the evolutionary search.
- Containerization: All system components will be containerized using Docker, enabling reproducible environments, execution isolation, and simplified deployment.

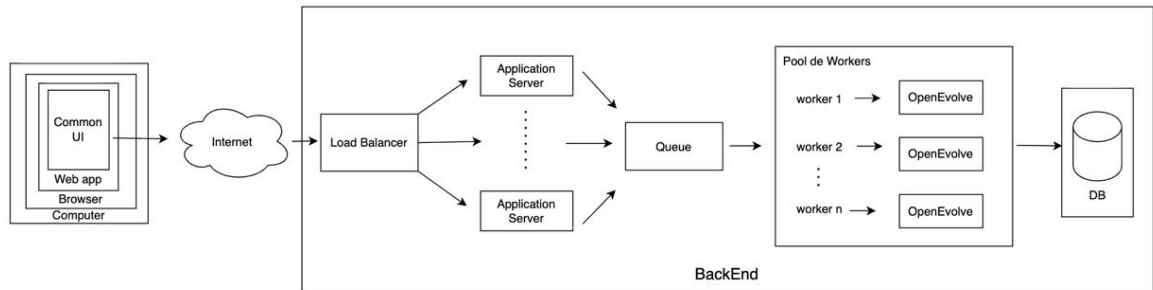


Figure 1: System Architecture

5 Risks

The development of the proposed platform involves several technical and operational risks:

- **OpenEvolve Integration** – The project requires integrating the OpenEvolve framework, which is implemented in Python. Given the limited prior experience with Python and the use of an unfamiliar external library, this may introduce a learning curve and integration challenges.
- **LLM Usage Cost** – The system relies on LLMs for solution generation. If not carefully managed, token-based usage may lead to increased operational costs.
- **Local LLM Integration** – Running or integrating local LLM models may introduce additional challenges related to configuration, hardware requirements, and system integration.

While these risks are manageable, they require careful architectural decisions, controlled resource management, and incremental validation during development.

6 Project Plan

Task	Duration	Deadline
Project proposal	1 week	March 9
OpenEvolve study and experimentation	1 week	March 16
System architecture and database design	1 week	March 23
Backend development	2 weeks	April 6
OpenEvolve integration	1 week	April 13
Frontend development	2 weeks	April 27
Progress milestone	1 week	April 27
System integration and improvements	2 weeks	May 11
Deployment in a Cloud provider	2 week	May 25
Beta version release	2 weeks	June 1
Testing and bug fixing	3 weeks	June 22
Report writing and project completion	3 weeks	July 11

Table 1: Project Plan with major MileStones

7 References

1. Audrey Cheng et al., Barbarians at the Gate: How AI is Upending Systems Research. <https://arxiv.org/abs/2510.06189> (visitado a 6-fev-2026)
2. Repositório Github, OpenEvolve: An Open-Source Evolutionary Coding Agent. <https://github.com/algorithmicsuperintelligence/openevolve> (visitado a 6-fev-2026)